Católica SC
Centro Universitário

ESTRUTURAS DE DADOS

Desafios de Code Interview

Ferramenta de IA utilizada: ChatGPT

Alunos
Dereck Conink
Geovania Francisco
Pâmela Baron

Ferramenta de IA Escolhida

Ferramenta: ChatGPT

Forma de acesso: Interface web, sem necessidade de instalação ou plugin.

Por que foi escolhida: O ChatGPT foi escolhido por possuir explicações simples e diretas, o que foi o melhor caminho para a equipe aprender enquanto resolvia os problemas.

Além disso, ele ajudou nos casos de teste de forma bem eficiente.

Não foi integrado ao VSCode.

Como foi utilizado

- Solicitação de explicações sobre algoritmos e estruturas de dados;
- Revisão de casos que causaram erros;
- Sugestão de ideias de testes;

Arrays

Problema 1 - Soma de Dois Números

Plataforma: LeetCode | **Estrutura:** Array + Dicionario Hash Map | **Dificuldade:** Facil

Link: https://leetcode.com/problems/two-sum/

Justificativa:
O array foi utilizado para armazenar os números de entrada, enquanto o dicionário hash map permitiu armazenar os valores já percorridos junto com seus índices. Isso possibilitou verificar rapidamente se o complemento necessário para atingir o alvo já foi encontrado. Essa forma evita o uso de dois loops aninhados, tornando a solução mais eficiente.

Escolha da IA e Uso:

Para que foi usada a IA:
Foi utilizada para validar a lógica da solução, sugerir melhorias de eficiência e auxiliar na criação de casos de teste.
Partes boas
Ajudou a entender a lógica do problema e sugeriu uma solução com dicionário.
Partes ruins
Pode levar à dependência, dificultando o desenvolvimento do raciocínio lógico próprio
O que a equipe ganhou
Rapidez na revisão e aprendizado de uma abordagem com melhor complexidade.
O que a equipe perdeu

Menor tempo de revisão prática pensando em soluções alternativas manualmente.

Estratégia utilizada

Ideia principal:

Percorrer o array uma única vez e, para cada número, verificar se já existe no dicionário o valor complementar necessário para atingir o alvo.

Por que a estrutura é adequada:

O dicionário permite buscas em tempo constante, tornando o algoritmo muito mais eficiente do que usar dois loops.

Casos especiais:

- Números repetidos
- Quando não existe solução
- Números negativos

Complexidade

De tempo: O(n)

De memória: O(n)

Problema 2 - Subarray de Soma Maxima

Plataforma: LeetCode | **Estrutura**: Array | **Dificuldade**: Médio

Link: https://leetcode.com/problems/maximum-subarray/

Justificativa:
O problema utilizou um array como estrutura principal, pois os dados são sequenciais e precisam ser percorridos uma única vez. A solução aplica o algoritmo de Kadane, que calcula a soma máxima de subarrays, mantendo apenas variáveis auxiliares, sem necessidade de estruturas adicionais

Escolha da IA e Uso:

Para que foi usada a IA:
A IA foi utilizada para compreender e aplicar o algoritmo de Kadane e sugerir testes variados.

Partes boas
Facilita o entendimento de algoritmos.

Partes ruins
Pode ocultar a lógica por trás da tomada de decisão do algoritmo.

O que a equipe ganhou
Aprendizado de uma solução eficiente em tempo linear.

O que a equipe perdeu
Menos prática em desenvolver a solução do zero.

Estratégia utilizada

Ideia principal:
Decidir a cada elemento se é melhor continuar o subarray atual ou iniciar um novo.
Por que a estrutura é adequada:
O array permite acesso sequencial necessário para aplicar o algoritmo.
Casos especiais:

- Todos negativos
- Apenas um elemento
- Todos positivos

Complexidade

De tempo: O(n)

De memória: O(1)

Problema 3 - Produto do Array Exceto Si Mesmo

Plataforma: LeetCode | **Estrutura**: Array | **Dificuldade**: Medio

Link: https://leetcode.com/problems/product-of-array-except-self/

Justificativa:
O array armazenou tanto os valores de entrada quanto o resultado. A solução utilizou duas passagens no array para calcular produtos acumulados à esquerda e à direita de cada elemento. Isso eliminou a necessidade de divisão e evitou estruturas extras complexas.

Escolha da IA e Uso:

Para que foi usada a IA:
A IA foi utilizada para entender a abordagem sem divisão e revisar a lógica das duas passagens no array.
Partes boas

- Explicou a técnica de prefixo e sufixo de forma muito clara
- Destacou corretamente que divisão nao e permitida pelo enunciado
- Gerou bons casos de teste incluindo o caso com zero

Partes ruins
Pode dificultar o entendimento da lógica se não houver estudo adicional.
O que a equipe ganhou
Conhecimento da técnica prefixo e sufixo.
O que a equipe perdeu
Menos resolução por métodos próprios.

Estratégia utilizada

Ideia principal:
Calcular o produto acumulado à esquerda e à direita de cada posição.
Por que a estrutura é adequada:
O array permite armazenar resultados intermediários e percorrer em ambas as direções.
Casos especiais:

- Zero no array onde todas as posições ficam com 0, exceto a posição do zero que recebe o produto dos demais
- Números negativos
- Pequeno número de elementos

Complexidade

De tempo: O(n)

De memória: O(n)

Problema 4 - Melhor Momento para Comprar e Vender Ação

Plataforma: LeetCode | **Estrutura**: Array | **Dificuldade**: Facil

Link: https://leetcode.com/problems/best-time-to-buy-and-sell-stock/

Justificativa:
O array foi utilizado para armazenar os preços das ações ao longo do tempo. A solução percorre o array apenas uma vez, mantendo o menor preço encontrado até o momento e calculando o lucro máximo possível. Esse modo é eficiente pois evita comparações desnecessárias.

Escolha da IA e Uso:

Para que foi usada a IA:
A IA foi utilizada para validar a lógica da solução e sugerir cenários de teste.
Partes boas
Ajudou a identificar soluções eficientes rapidamente.
Partes ruins
Reduzir o esforço de raciocínio.
O que a equipe ganhou
Clareza na estratégia e otimização do código.
O que a equipe perdeu
Menor prática com soluções menos eficientes.

Estratégia utilizada

Ideia principal:
Acompanhar o menor preço e calcular o maior lucro possível ao longo do tempo.
Por que a estrutura é adequada:
O array representa os preços ao longo do tempo, permitindo análise sequencial.
Casos especiais:

- Preços sempre em queda
- Apenas um elemento
- Preços constantes

Complexidade

De tempo: O(n)

De memória: O(1)

Problema 5 - Fim do Array Mínimo

Plataforma: LeetCode | **Estrutura**: Array | **Dificuldade**: Médio

Link: https://leetcode.com/problems/minimum-array-end/description/

Justificativa:

O problema não exige a construção explícita do array, mas sim encontrar o menor valor possível do último elemento. A solução utiliza operações bit a bit para garantir que:

- Todos os elementos do array tenham AND igual a x
- O array seja estritamente crescente
- O último valor seja o menor possível

A ideia é usar os bits que são 0 em x como "espaços livres" para inserir os bits de n - 1, garantindo crescimento mínimo sem quebrar o AND. Essa abordagem é eficiente porque evita gerar o array completo e trabalha diretamente na construção do resultado.

Escolha da IA e Uso:

Para que foi usada a IA:
A IA foi utilizada para validar a lógica com operações bit a bit e auxiliar na simplificação da implementação.
Partes boas
Ajudou a encontrar rapidamente uma solução ótima (bitwise)
Facilitou a visualização da estratégia sem construir o array
Partes ruins
Pode reduzir a prática de descoberta manual da solução
A lógica com bits pode parecer menos intuitiva inicialmente
O que a equipe ganhou
Melhor entendimento de manipulação de bits
Código mais eficiente e direto
O que a equipe perdeu
Menor exploração de soluções intermediárias (menos eficientes).

Estratégia utilizada

Ideia principal:
Construir o menor valor possível de nums[n-1] inserindo os bits de n-1 nas posições onde x possui bits 0.
Por que a estrutura é adequada:
Não precisamos de array
Trabalhamos diretamente com inteiros (bitwise)
Garante crescimento e AND correto simultaneamente
Casos especiais:

- n = 1 -> resposta é o próprio x
- x com todos os bits 1 -> precisamos usar bits mais altos
- n grande -> ainda funciona eficientemente (bit a bit)

Complexidade

De tempo: O(log n)

De memória: O(1)

Listas Encadeadas

Problema 1 - Inverter Lista Encadeada

Plataforma: LeetCode | **Estrutura**: Lista Encadeada | **Dificuldade**: Facil

Link: https://leetcode.com/problems/reverse-linked-list/

Justificativa:
A lista encadeada foi utilizada pois o problema envolve manipulação direta de ponteiros entre os nós. A solução atuou invertendo as ligações entre os elementos, sem necessidade de criar uma nova estrutura. Isso tornou o algoritmo eficiente em termos de memória, pois apenas altera as referências existentes.

Escolha da IA e Uso:

Para que foi usada a IA:
Foi utilizada para validar a lógica de manipulação de ponteiros e sugerir casos de teste.
Partes boas

- Explicou o conceito de inversão de ponteiros.
- Sugeriu testes importantes como lista vazia e lista com um elemento.

Partes ruins

- Pode dificultar o entendimento da lógica de ponteiros se não houver prática.
- A abstração pode esconder detalhes importantes da manipulação da memória.

O que a equipe ganhou

- Melhor compreensão da manipulação de listas encadeadas.
- Maior segurança na implementação.

O que a equipe perdeu
Menos prática na construção da solução sem auxílio.

Estratégia utilizada

Ideia principal:
Percorrer a lista e inverter os ponteiros de cada nó, fazendo com que cada elemento passe a apontar para o anterior.
Por que a estrutura é adequada:
A lista encadeada permite alterar diretamente as referências entre os nós, possibilitando a inversão sem necessidade de memória extra.
Casos especiais:

- Lista vazia
- Lista com apenas um elemento
- Lista com dois elementos

Complexidade
De tempo: O(n)

De memória: O(1)

Problema 2 - Mesclar Duas Listas Ordenadas

Plataforma: LeetCode | **Estrutura**: Lista Encadeada | **Dificuldade**: Facil

Link: https://leetcode.com/problems/merge-two-sorted-lists/

Justificativa:
A lista encadeada foi utilizada pois o problema requer a reorganização de elementos que já estão ordenados em duas sequências distintas. A solução atua comparando os nós de ambas as listas e ajustando os ponteiros para criar uma única sequência linear, sem a necessidade de alocar novos nós para os valores já existentes. Isso garante eficiência de memória, manipulando apenas as referências de cada elemento.

Escolha da IA e Uso:

Para que foi usada a IA:
Foi utilizada para validar a concatenação final de listas de tamanhos diferentes.
Partes boas

- Ajudou a simplificar a lógica do início da lista com o uso do nó auxiliar.
- Garantiu que o restante de uma lista não fosse perdido quando a outra terminasse primeiro.

Partes ruins
Exige atenção para não criar loops infinitos ao manipular as referências.
O que a equipe ganhou

- Conhecimento sobre a técnica de ponteiro auxiliar para construção de listas.
- Entendimento de como manter a ordenação original durante a fusão.

O que a equipe perdeu
Menos tempo raciocinando sobre o caso.

Estratégia utilizada

Ideia principal:
Utilizar um nó sentinela para iniciar a lista resultante e percorrer ambas as listas simultaneamente, escolhendo sempre o menor valor para conectar ao ponteiro atual.
Por que a estrutura é adequada:
Permite costurar as duas listas originais em uma nova ordem apenas alterando os ponteiros, mantendo a complexidade de espaço.
Casos especiais:

- Ambas as listas vazias (retorna None).
- Uma lista vazia e outra populada (retorna a lista populada).
- Listas com valores alternados ex: [1,3,5] e [2,4,6].

Complexidade

De tempo: O(n + m)

De memória: O(1)

Problema 3 - Ciclo em Lista Encadeada

Plataforma: LeetCode | **Estrutura**: Lista Encadeada | **Dificuldade**: Facil

Link: https://leetcode.com/problems/linked-list-cycle/

Justificativa:
A estrutura de lista encadeada é a base do problema, pois ciclos só ocorrem quando um nó aponta para um elemento anterior da mesma sequência, algo impossível em arrays tradicionais sem índices. A detecção de ciclo válida a integridade da estrutura, garantindo que o percurso tenha um fim definido.

Escolha da IA e Uso:

Para que foi usada a IA:
Foi utilizada para implementar o Algoritmo de Floyd e sugerir a criação de funções de teste que forçam a criação de ciclos em memória.
Partes boas
Gerou a função auxiliar de criação de ciclo para testes, o que foi muito útil.
Partes ruins
A explicação de porque os ponteiros se encontram ficou vaga.
O que a equipe ganhou
Conhecimento do algoritmo de Floyd.
O que a equipe perdeu
A oportunidade de tentar resolver usando estruturas adicionais.

Estratégia utilizada

Ideia principal:
Utilizar dois ponteiros com velocidades diferentes; se houver um ciclo, o ponteiro rápido eventualmente encontrará o ponteiro lento por trás.
Por que a estrutura é adequada:
A detecção depende puramente da navegação entre referências de memória, que é a característica principal das listas encadeadas.
Casos especiais:

- Lista vazia ou com apenas um nó sem ciclo.
- Ciclo no primeiro nó.
- Lista muito grande com ciclo apenas no final.

Complexidade

De tempo: O(n)

De memória: O(1)

Problema 4 - Lista Encadeada de Design

Plataforma: LeetCode | **Estrutura**: Lista Encadeada | **Dificuldade**: Médio

Link: https://leetcode.com/problems/design-linked-list/description/

Justificativa:
O problema exige a implementação manual de uma lista encadeada, sem utilizar bibliotecas prontas. Isso é importante para compreender profundamente como estruturas

dinâmicas funcionam internamente. A implementação permite controlar diretamente inserções em posições específicas e remoções e navegação entre nós. Além disso, o problema reforça conceitos fundamentais como ponteiros/referências, manipulação de memória lógica e estruturas dinâmicas

Escolha da IA e Uso:

Para que foi usada a IA:
Foi utilizada para validar a estrutura da implementação, sugerir melhorias (uso de lista duplamente encadeada), e ajudar na organização do código.
Partes boas
Ajudou a estruturar melhor a solução e tornou o código mais limpo e reutilizável.
Partes ruins
Reduz o esforço de depuração, que é importante no aprendizado e diminuiu a prática de implementação manual do zero.
O que a equipe ganhou
Melhor entendimento de listas encadeadas e código mais robusto e organizado
O que a equipe perdeu
Menos prática na identificação de erros estruturais e redução do processo de tentativa e erro

Estratégia utilizada

Ideia principal:
Implementar uma lista duplamente encadeada com nós sentinela (head e tail), permitindo inserções e remoções eficientes sem necessidade de tratar múltiplos casos especiais.
Por que a estrutura é adequada:
Sentinelas eliminam casos especiais (lista vazia, início/fim) e operações como inserção e remoção tornam-se mais simples e seguras
Casos especiais:
Índice inválido ou negativo
Inserção em índice 0 ou no tamanho da lista
Lista vazia
Complexidade

De tempo: O(n), O(1)

Memória: O(n)

Pilhas

Problema 1 - Parenteses Validos

Plataforma: LeetCode | **Estrutura**: Pilha Stack | **Dificuldade**: Facil

Link: https://leetcode.com/problems/valid-parentheses/

Justificativa:
A pilha é a estrutura ideal para este problema devido à necessidade de processar símbolos aninhados. Em expressões como {[()]}, o último símbolo de abertura lido deve ser o primeiro a ser fechado. A pilha armazena os caracteres de abertura e permite

verificar, a cada símbolo de fechamento, se ele corresponde ao topo da pilha, respeitando a ordem de hierarquia e fechamento.

Escolha da IA e Uso:

Para que foi usada a IA:
Foi utilizada para otimizar a verificação dos pares usando um dicionário e para definir os casos de erro, como pilhas vazias prematuramente.
Partes boas

- Sugeriu o uso de um dicionário para tornar o código mais limpo e fácil de expandir para novos símbolos.
- Identificou a necessidade de checar se a pilha está vazia ao final do processo (len(pilha) == 0).

Partes ruins
Pode levar à implementação mecânica sem entender por que o caractere de fechamento não entra na pilha.
O que a equipe ganhou

- Código mais conciso e performático.
- Melhor entendimento sobre como pilhas são usadas em compiladores para validar sintaxe.

O que a equipe perdeu
A experiência de debugar manualmente erros de índice fora de alcance ao tentar acessar o topo de pilhas vazias.

Estratégia utilizada

Ideia principal:
Percorrer a string e empilhar símbolos de abertura. Ao encontrar um fechamento, verificar se ele confere com o topo da pilha; se sim, remove-se o topo; se não ou se a pilha estiver vazia, a string é inválida.
Por que a estrutura é adequada:
A natureza Last In, First Out da pilha garante que o fechamento verificado seja sempre referente ao símbolo de abertura mais recente, o que é a definição de aninhamento correto.
Casos especiais:

- String iniciada por símbolo de fechamento.
- String com apenas símbolos de abertura.
- Símbolos de tipos diferentes que se cruzam ex: ([)].

Complexidade

De tempo: O(n)

De memória: O(n)

Problema 2 -Pilha com Mínimo

Plataforma: LeetCode | **Estrutura**: Pilha Stack | **Dificuldade**: Médio

Link: https://leetcode.com/problems/min-stack/

Justificativa:

A estrutura de pilha foi utilizada para garantir a ordem Last In, First Out, fundamental para o funcionamento de uma pilha comum. O desafio era obter o valor mínimo em tempo constante O(1). Para isso, utilizou-se uma estratégia de pilha dupla: uma armazena os valores reais e a outra rastreia o valor mínimo correspondente a cada nível da pilha principal. Essa abordagem evita a necessidade de percorrer a lista inteira toda vez que o mínimo é solicitado.

Escolha da IA e Uso:

Para que foi usada a IA:
Foi utilizada para validar a lógica de sincronização entre as duas pilhas e sugerir casos de teste onde o mínimo permanece estável mesmo após remoções.
Partes boas

- Ajudou a definir a lógica de inserção, comparar o novo valor com o topo da pilha de mínimos.
- Sugeriu cenários de teste, como a remoção do próprio elemento mínimo.

Partes ruins

- O uso de duas pilhas dobra o consumo de memória, o que pode não ser intuitivo inicialmente.
- Requer cuidado extremo para que as operações de push e pop ocorram sempre em ambas as estruturas simultaneamente.

O que a equipe ganhou

- Compreensão de como otimizar consultas O(1) sacrificando um pouco de espaço em memória.
- Implementação robusta de uma estrutura de dados customizada.

O que a equipe perdeu
Menos esforço na descoberta da técnica de espelhamento de pilhas.

Estratégia utilizada

Ideia principal:
Manter uma pilha auxiliar pilha_minimos onde cada elemento no topo representa o menor valor presente na pilha_principal até aquele momento.
Por que a estrutura é adequada:
A pilha permite acesso rápido ao último elemento inserido. Ao espelhar o mínimo, garantimos que, ao desempilhar um valor, o mínimo anterior seja restaurado instantaneamente no topo da pilha auxiliar.
Casos especiais

- Pilha com apenas um elemento, o elemento é o próprio mínimo.
- Sequências onde o mínimo não muda após várias inserções ou remoções.
- Remoção do elemento que é o atual mínimo global.

Complexidade

De tempo: O(1)

De memória: O(n)

Problema 3 - Pegue os presentes da pilha mais rica

Plataforma: LeetCode | **Estrutura**: Pilha Stack | **Dificuldade**: Fácil

Link: https://leetcode.com/problems/take-gifts-from-the-richest-pile/description/

Justificativa:
A estrutura de heap máximo foi utilizada para garantir acesso rápido ao maior elemento em cada operação, fundamental para o funcionamento eficiente do problema. O desafio era reduzir repetidamente o maior valor para sua raiz quadrada arredondada para baixo e manter a ordenação do heap sem percorrer todo o array a cada operação. Essa abordagem evita a necessidade de buscas lineares do máximo, garantindo eficiência mesmo para k operações consecutivas.

Escolha da IA e Uso:

Para que foi usada a IA:
Foi utilizada para validar a lógica de remoção e reinserção do maior elemento, sugerir a estrutura de heap máxima e propor casos de teste onde múltiplos elementos têm o mesmo valor máximo.
Partes boas
Ajudou a definir a lógica de atualização do heap de forma correta após cada operação, e sugeriu cenários de teste, como presentes iguais ou presentes com valor 1, garantindo que a função se comporta corretamente.
Partes ruins
Usar heap pode ser menos intuitivo do que percorrer o array manualmente para iniciantes, e o conceito de inserir valores negativos para simular max-heap em Python exige atenção extra.
O que a equipe ganhou
Compreensão de como usar heap para acessar e atualizar rapidamente elementos máximos, e implementação eficiente em O(n + k log n), robusta para diferentes tamanhos de entrada.
O que a equipe perdeu
Menos exploração de soluções de força bruta que percorrem o array manualmente, reduzindo prática em lógica linear simples.

Estratégia utilizada

Ideia principal:
Transformar o array de presentes em um max-heap. A cada segundo, retirar o maior elemento, reduzir para sua raiz quadrada arredondada para baixo e reinserir no heap. Após k operações, somar todos os elementos restantes para obter o total de presentes.

Por que a estrutura é adequada:
O heap permite acesso O(1) ao maior elemento e atualização O(log n) após cada operação, garantindo eficiência. Sem heap, seria necessário percorrer todo o array a cada passo, aumentando a complexidade para O(k·n).

Casos especiais

- k = 0: não há operações, soma dos presentes permanece igual.
- Todos os presentes iguais: o heap mantém a estabilidade e as operações ocorrem corretamente.

- Presentes com valor 1: a raiz quadrada é 1, o valor não muda, e o heap continua consistente.

Complexidade

De tempo: O(n + k log n)

De memória: O(n)

Como Usei IA Nesta Atividade

Qual IA foi usada?
Chat GPT

Prompt 1:

tenho esse codigo do leetcode para resolver mas nao estou entendendo, voce pode me explicar??

Given an integer array nums, return an array answer such that answer[i] is equal to the product of all the elements of nums except nums[i].
The product of any prefix or suffix of nums is guaranteed to fit in a 32-bit integer.
You must write an algorithm that runs in O(n) time and without using the division operation.

Example 1:
Input: nums = [1,2,3,4]Output: [24,12,8,6]
Example 2:
Input: nums = [-1,1,0,-3,3]Output: [0,0,9,0,0]

Constraints:
2 <= nums.length <= 105
-30 <= nums[i] <= 30
The input is generated such that answer[i] is guaranteed to fit in a 32-bit integer.

Follow up: Can you solve the problem in O(1) extra space complexity? (The output array does not count as extra space for space complexity analysis.)

Por que a equipe perguntou isso?
A equipe recorreu à IA pois precisava trabalhar com problemas de nível médio, mas necessitava de auxílio para compreendê-los.

O que a IA respondeu?

Disse que a solução utiliza a técnica de prefixos e sufixos, dividindo o problema em duas passagens pelo array.

Na primeira, percorre da esquerda para a direita, calculando o produto acumulado dos elementos anteriores = prefixos.

A Lógica dos Prefixos e Sufixos

Podemos dividir o problema em duas passagens pelo array:

1. **Passagem da Esquerda para a Direita (Prefixos):** Calculamos o produto de todos os elementos antes de i.
2. **Passagem da Direita para a Esquerda (Sufixos):** Calculamos o produto de todos os elementos depois de i e multiplicamos pelo que já tínhamos.

Exemplo Passo a Passo: `nums = [1, 2, 3, 4]`

1. Calculando os Prefixos:

Criamos um array `res` onde cada posição guarda a multiplicação dos vizinhos da esquerda.

- `res[0] = 1` (não tem nada antes do primeiro elemento).
- `res[1] = res[0] * nums[0]` $\rightarrow 1 \times 1 = 1$
- `res[2] = res[1] * nums[1]` $\rightarrow 1 \times 2 = 2$
- `res[3] = res[2] * nums[2]` $\rightarrow 2 \times 3 = 6$
- *Resultado parcial:* `[1, 1, 2, 6]`

2. Calculando os Sufixos (e finalizando):

Agora voltamos do final para o começo, mantendo uma variável acumuladora (vamos chamá-la de `post`).

- Começamos com `post = 1`.
- `res[3] = res[3] * post` $\rightarrow 6 \times 1 = 6$. Atualizamos `post = post * nums[3]` ($1 \times 4 = 4$).

2. Calculando os Sufixos (e finalizando):

Agora voltamos do final para o começo, mantendo uma variável acumuladora (vamos chamá-la de `post`).

- Começamos com `post = 1`.
- `res[3] = res[3] * post` $\rightarrow 6 \times 1 = 6$. Atualizamos `post = post * nums[3]` ($1 \times 4 = 4$).
- `res[2] = res[2] * post` $\rightarrow 2 \times 4 = 8$. Atualizamos `post = post * nums[2]` ($4 \times 3 = 12$).
- `res[1] = res[1] * post` $\rightarrow 1 \times 12 = 12$. Atualizamos `post = post * nums[1]` ($12 \times 2 = 24$).
- `res[0] = res[0] * post` $\rightarrow 1 \times 24 = 24$.
- *Resultado final:* `[24, 12, 8, 6]`

O que foi aceito?

Utilizar duas passagens pelo array, prefixo e sufixo, foi mantida, assim como a ideia de evitar o uso de divisão.

O que precisou ser corrigido ou adaptado?

A equipe reorganizou a lógica para percorrer o array completo na primeira passagem em vez de iniciar no índice 1, tornando o código mais consistente e fácil de entender.

Também foram incluídos comentários explicativos para facilitar a leitura e manutenção do código.

Visão final

O ChatGPT apresentou corretamente a lógica geral da solução, mas em alguns momentos simplificou a explicação, o que poderia dificultar o entendimento completo da técnica para quem ainda está aprendendo. Em alguns pontos, a explicação foi superficial, principalmente na forma como os prefixos e sufixos se combinam, exigindo que a equipe analisasse o código com mais atenção para compreender o funcionamento real.

Prompt 2:

Quando eu tento na função de inverter a lista, fazer atual.proximo = anterior, logo de primeira da erro e nao entendo o motivo.

Promoção gratuita ×

quando eu tento na função de inverter a lista, fazer atual.proximo = anterior logo de prmeira? da erro e nao entendo pq# Definicao do No da lista encadeada

```
class No:
    def __init__(self, valor=0, proximo=None):
        self.valor = valor
        self.proximo = proximo
```

Por que a equipe perguntou isso?

Pois foi tentado simplificar o código mas acabou gerando problema.

O que a IA respondeu?

Você perde a referência do resto da lista e não tem mais como continuar o loop

💥 Por que dá erro se fizer direto?

Se você fizer isso logo de primeira:

Python Executar

```python
atual.proximo = anterior
```

sem salvar antes:

Python Executar

```python
proximo = atual.proximo
```

👉 você **perde a referência do resto da lista**

O que foi aceito?

A lógica de inversão da lista encadeada foi mantida, juntamente com o uso de três ponteiros anterior, atual e próximo para controlar a navegação e evitar a perda de referência dos nós.

O que precisou ser corrigido ou adaptado?

Nós organizamos melhor os nomes das variáveis e adicionamos comentários para tornar o código mais compreensível.

Visão final

A IA apresentou corretamente a lógica geral da inversão da lista encadeada, e deixou totalmente explícita a importância da ordem das operações.

Prompt 3:

Preciso testar o script da lista encadeada de design, nível médio.

Preciso testar o script da lista encadeada de design, nível médio.

Perfeito! Vamos criar **testes completos para a sua lista encadeada de design**, seguindo o problema do LeetCode "Design Linked List". O objetivo é testar todas as operações: `addAtHead`, `addAtTail`, `addAtIndex`, `get` e `deleteAtIndex`.

Vou fornecer um **script completo que você pode rodar diretamente**.

🖥️ Script de testes para `MyLinkedList`

Python Executar

```python
# Classe de nó para lista encadeada simples
class Node:
    def __init__(self, val):
        self.val = val
        self.next = None
```

Por que a equipe perguntou isso?

A equipe recorreu à IA pois precisava testar o código que foi desenvolvido, e verificar o resultado esperado pelo script em Python.

O que a IA respondeu?

A partir da lista encadeada "MyLinkedList" , foram criados testes para cada operação: addAtHead, addAtTail, addAtIndex, get e deleteAtIndex.

O que foi aceito?

Casos de borda: índice negativo, índice maior que tamanho, deletar da cabeça e da cauda.

O que precisou ser corrigido ou adaptado?

Foram incluídos comentários explicativos para facilitar a leitura e manutenção do código.

Visão final

O ChatGPT criou os testes e colocou os resultados esperados corretamente para o script de lista encadeada de design.

Prompt 4 :

preciso de testes para o script da Testes de class Solution:

preciso de testes para o script da Testes de class Solution:

Perfeito! Vamos criar **testes completos e variados** para a sua classe `Solution` com o método `pickGifts`. Vou incluir casos do enunciado, casos limite e alguns adicionais para garantir que seu código funciona em qualquer situação.

🖥️ Script de testes completo

Python Executar

```python
import heapq
import math
from typing import List

# Classe fornecida
class Solution:
    def pickGifts(self, gifts: List[int], k: int) -> int:
        # Transformar a lista de presentes em um max-heap
        max_heap = [-gift for gift in gifts]
        heapq.heapify(max_heap)
```

Por que a equipe perguntou isso?

A equipe recorreu à IA pois precisava testar a classe Solution, do problema “3- Pegar o presente da pilha mais rica”, do leetcode.

O que a IA respondeu?

Perfeito! Vamos criar testes completos e variados para a sua classe Solution com o método pickGifts. Vou incluir casos do enunciado, casos limite e alguns adicionais para garantir que seu código funciona em qualquer situação.

O que foi aceito?

Implementação do max-heap usando números negativos para pegar sempre o maior presente.

O que precisou ser corrigido ou adaptado?

O teste ajudou a detectar erros de ponteiro na classe Solution.

Visão final

O ChatGPT criou os testes e colocou os resultados esperados corretamente para o script de lista encadeada de design.

Prompt 5:

“Você pode me explicar a complexidade por traz desse código do leetcode?”

voce pode me explicar a complexidade por traz desse codigo do leetcode?

```
class PilhaComMinimo:

    def __init__(self):
        self.pilha_principal = []
        self.pilha_minimos   = []

    def empilhar(self, valor):
        self.pilha_principal.append(valor)

        # O minimo atual e o menor entre o novo valor e o minimo
anterior
        if self.pilha_minimos:
            minimo_atual = min(valor, self.pilha_minimos[-1])
        else:
            minimo_atual = valor  # Primeira insercao: o proprio valor e o
```

Por que a equipe perguntou isso?

Porque precisava compreender a complexidade de tempo e memória do algoritmo, já que o problema envolve duas estruturas simultâneas de pilha principal e pilha de mínimos. Além disso, era necessário justificar por que o uso de duas pilhas torna possível obter o mínimo em tempo constante.

O que a IA respondeu?

Explicou que o código funciona usando duas pilhas, uma para armazenar os valores e outra para armazenar o menor valor até cada posição. Também foi explicado que todas as operações de empilhar, desempilhar, topo e mínimo, possuem tempo constante O(1), pois realizam apenas operações simples como append, pop e acesso ao último elemento. Ela também informou que a complexidade de memória é O(n), já que são armazenados n elementos em cada pilha.

✅ Estrutura usada

- `pilha_principal` : guarda todos os valores empilhados normalmente.
- `pilha_minimos` : guarda, em cada posição, o **menor valor até aquele momento.**

Ou seja:

- Se eu empilho 5, depois 2, depois 8
- a pilha principal fica: `[5, 2, 8]`
- a pilha de mínimos fica: `[5, 2, 2]`

Assim, o mínimo sempre está no topo da pilha de mínimos.

⏱️ Complexidade de tempo (por operação)

`empilhar(valor)` → **O(1)**

- `append` na lista é O(1)
- pegar o topo `self.pilha_minimos[-1]` é O(1)
- `min(valor, topo)` é O(1) (comparação simples)
- `append` de novo é O(1)

➡️ Então empilhar é constante.

`desempilhar()` → **O(1)**

- `pop()` na lista é O(1)
- faz isso em duas pilhas

➡️ Então desempilhar é constante.

O que foi aceito?

Foi aceito que todas as operações do algoritmo possuem complexidade O(1), pois nenhuma delas depende do tamanho da pilha para ser executada. Também foi aceito que o consumo de memória é O(n), já que as duas pilhas crescem proporcionalmente ao número de elementos inseridos.

O que precisou ser corrigido ou adaptado?

Foi necessário adaptar a implementação para garantir que o método minimo() retornasse o menor valor em tempo constante, sem precisar percorrer toda a pilha principal. Para isso, foi adicionada uma segunda pilha, a pilha_minimos, que armazena o menor valor acumulado até cada posição da pilha.

Também foi preciso ajustar a lógica do método empilhar() para que, a cada inserção, fosse comparado o novo valor com o mínimo anterior garantindo que o topo da pilha de mínimos sempre representasse o menor elemento atual.

Por fim, foi necessário garantir que no método desempilhar() os elementos fossem removidos simultaneamente das duas pilhas, evitando inconsistência entre a pilha principal e a pilha de mínimos.

Visão final

A IA não inventou informações e a princípio toda a explicação foi correta, porém foi superficial ao não detalhar que a operação append e pop são consideradas O(1) no Python por serem realizadas no final da lista.

Prompt 6:

"Me de ideias de testes para esse codigo:"

```
me de ideias de testes para esse codigo:

def parenteses_validos(texto):
    # Dicionario que mapeia cada fecha para o seu abre correspondente
    pares = {
        ')': '(',
        '}': '{',
        ']': '['
    }

    # A pilha vai guardar os simbolos de abertura encontrados
    pilha = []

    for caractere in texto:
        if caractere in pares:
            # E um simbolo de fechamento
            # Verificamos se o topo da pilha e o par correto
            if pilha and pilha[-1] == pares[caractere]:
                pilha.pop()  # Par correto: desempilhamos
            else:
                return False  # Par errado ou pilha vazia: invalido
        else:
            # E um simbolo de abertura: empilhamos
            pilha.append(caractere)

    # Se a pilha estiver vazia, todos os pares foram fechados
corretamente
    return len(pilha) == 0
```

Por que a equipe perguntou isso?

Porque era preciso criar testes para validar a função de parênteses válidos.

O que a IA respondeu?

Sugeriu vários tipos de testes, como casos válidos simples e múltiplos símbolos válidos. Também destacou que o código assume que o texto contém apenas símbolos de parênteses, colchetes e chaves, e que letras ou outros caracteres poderiam gerar resultados incorretos.

✅ Testes básicos (corretos)

- `"()"` → True
- `"{}"` → True
- `"[]"` → True
- `"(){}[]"` → True
- `"{[()]}"` → True
- `"((((()))))"` → True

❌ Testes de ordem errada (inválidos)

- `"(]"` → False
- `"{)"` → False
- `"[)"` → False
- `"([)]"` → False *(ordem incorreta, mesmo tendo os símbolos certos)*
- `"{[(])}"` → False

❌ Testes com fechamento sem abertura

- `")"` → False
- `"}"` → False
- `"]"` → False
- `")("` → False

⚠️ Observação importante sobre seu código

O seu código assume que **qualquer caractere que não seja** `)`, `]`, `}` **é abertura.**

Então se você testar:

- `"a"` → ele vai colocar `a` na pilha e retornar **False**
- `"(a)"` → ele vai empilhar `(` e `a`, e quando fechar `)` vai dar **False**

📌 Ou seja: esse código só funciona corretamente se a string tiver **apenas os caracteres** `(){}[]`.

O que foi aceito?

Foram aceitos os testes que cobrem os principais cenários necessários para validar o algoritmo:

- um par simples válido ("()")
- múltiplos tipos válidos ("()[]{}")
- tipos cruzados inválidos ("(]")
- aninhamento correto ("{[()]}")
- apenas aberturas sem fechamento ("(([")

O que precisou ser corrigido ou adaptado?

Foi necessário adaptar as sugestões para selecionar apenas os testes mais relevantes e objetivos. Além disso, foi importante incluir um caso em que existem apenas aberturas sem fechamento, pois esse é um erro comum e ajuda a demonstrar que o algoritmo só retorna verdadeiro quando a pilha termina vazia.

Visão final

A IA não inventou informações incorretas, pois os testes sugeridos eram coerentes com o funcionamento do algoritmo.

Prompt 7:

"Preciso de mais um teste para esse código"

```
preciso de mais um teste para esse codigo, me ajude:


class No:
    def __init__(self, valor=0, proximo=None):
        self.valor = valor
        self.proximo = proximo


def tem_ciclo(cabeca):
    # Lista vazia ou com unico no sem ciclo nao tem ciclo
    if cabeca is None or cabeca.proximo is None:
        return False

    tartaruga = cabeca          # Avanca 1 passo por vez
    lebre = cabeca.proximo      # Avanca 2 passos por vez

    while lebre is not None and lebre.proximo is not None:
        # Se os dois se encontrarem, ha ciclo
        if tartaruga == lebre:
            return True

        tartaruga = tartaruga.proximo       # 1 passo
        lebre = lebre.proximo.proximo       # 2 passos
```

Por que a equipe perguntou isso?

Porque precisava adicionar mais um teste para a validação do algoritmo de detecção de ciclo.

O que a IA respondeu?

Sugeriu como teste adicional o caso de uma lista com apenas um nó que aponta para ele mesmo, formando um ciclo mínimo possível.

Um teste muito bom que ainda falta é o caso de **lista com um único nó que aponta para ele mesmo** (ciclo no próprio nó).
Esse é um caso clássico e bem importante.

Coloque assim:

Python Executar

```python
# lista com um unico no com ciclo (aponta para ele mesmo)
lista_4 = criar_lista_com_ciclo([1], posicao_ciclo=0)
resultado_4 = tem_ciclo(lista_4)
print("Teste unico no com ciclo:")
print("Valores: [1], ciclo no indice 0")
print("Esperado: True | Obtido:",
print()
```

Perguntar ao ChatGPT

📌 Por que esse teste é importante?
Porque valida se o algoritmo detecta ciclo mesmo no menor caso possível, onde o nó fecha nele mesmo.

O que foi aceito?

A ideia de teste foi aceita, mas alterada para uma forma mais simples.

O que precisou ser corrigido ou adaptado?

A sugestão foi adaptada para uma forma mais simples, uma lista com apenas um nó sem ciclo, para validar a condição inicial do código, garantindo que o algoritmo retorna corretamente false quando não existe possibilidade de ciclo.

Visão final

A ideia do chat foi boa, mas a resposta foi fraca por não ter considerado que o código já trata listas de um nó automaticamente com a verificação inicial, o que poderia tornar o teste de ciclo em um nó menos relevante. Foi preciso decidir por conta própria qual caso seria melhor para reforçar a confiabilidade do código, optando pelo teste de lista com um único nó sem ciclo, por ser mais alinhado com a validação do comportamento esperado.

Prompt 8:

"Explique o conceito de Complexidade de tempo e memória, exemplo O(n), O(1) .... ainda nao ficou muito claro"

explique o conceito de Complexidade de tempo e memória, exemplo O(n), O(1) .... ainda nao ficou muito claro

Por que a equipe perguntou isso?

Pois ainda havia dúvida no conceito de complexidade de tempo e memória, principalmente a interpretação de notações como O(1), O(n) e $O(n^2)$. Como o trabalho exige justificar a eficiência dos algoritmos, era necessário entender o que essas notações significam e como aplicar nos códigos.

O que a IA respondeu?

Ela explicou que complexidade de tempo representa como o número de operações cresce conforme aumenta o tamanho da entrada (n), enquanto a complexidade de memória representa o quanto de memória adicional o algoritmo utiliza além dos dados de entrada. A IA apresentou exemplos simples para O(1), O(n) e O(n^2), relacionando cada um com estruturas como loops simples e loops aninhados.

✅ O que é Complexidade?

Quando a gente fala em **complexidade**, estamos respondendo duas perguntas:

⏱️ **1) Complexidade de tempo**

➡️ **Quanto tempo o algoritmo demora** para rodar conforme a entrada cresce.

💾 **2) Complexidade de memória**

➡️ **Quanta memória extra** o algoritmo precisa conforme a entrada cresce.

O importante é:

📌 **não é o tempo em segundos**, é como o tempo cresce quando o tamanho do input aumenta.

🔥 O que significa esse "n"?

n significa:

👉 **quantidade de elementos de entrada**

Exemplo:

- lista com 5 números → n = 5
- lista com 100 números → n = 100

📌 Complexidade de tempo: exemplos

🟢 **O(1) — Tempo constante**

Significa que o algoritmo faz sempre a mesma quantidade de trabalho, não importa o tamanho.

Exemplo:

Python Executar

```python
def pegar_primeiro(lista):
    return lista[0]
```

Se a lista tiver:

- 5 elementos
- 5000 elementos

Ele só pega o primeiro, então demora praticamente igual.

➡️ **O(1) = constante**

🧠 Resumo bem fácil de memorizar

Tempo:

- O(1) → faz sempre a mesma coisa
- O(n) → percorre tudo uma vez
- O(n^2) → percorre tudo várias vezes (loop dentro de loop)

Memória:

- O(1) → só usa variáveis
- O(n) → cria lista/dicionário auxiliar que cresce com a entrada

O que foi aceito?

Foi aceito que a explicação com exemplos práticos tornou o conceito mais compreensível. Também foi aceito o entendimento de que O(1) significa tempo ou memória constante, independentemente do tamanho do input.

O que precisou ser corrigido ou adaptado?

A equipe precisou decidir como resumir essas informações para que ficassem objetivas no relatório.

Visão final

A IA não inventou informações incorretas, pois os exemplos e definições apresentados estavam alinhados com o conceito correto de Big-O. A equipe precisou decidir por conta própria de que forma as informações ficariam mais adequados para incluir no trabalho.